Neural Networks

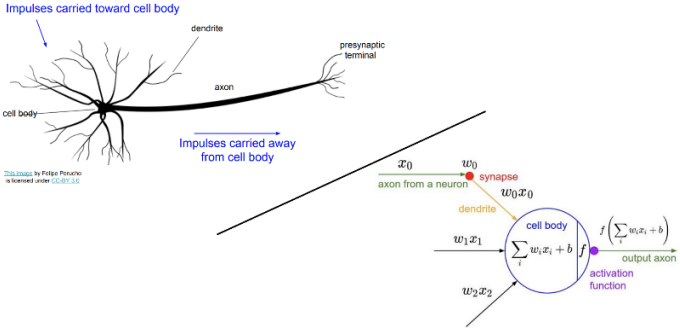
Thien Huynh-The

Department of Computer and Communications Engineering
HCMC University of Technology and Education

December 26, 2025

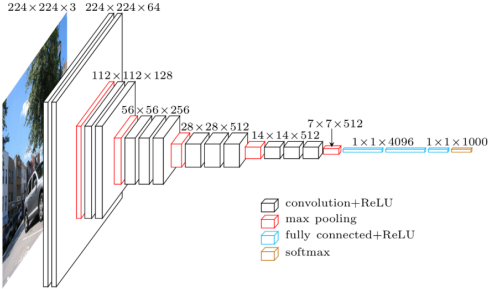
What are Neural Networks?

- Inspired by the biological neural networks in our brains.
- Powerful machine learning models capable of learning complex patterns from data.
- Consist of interconnected nodes (neurons) organized in layers.
- Used for various tasks: classification, regression, image recognition, natural language processing, etc.

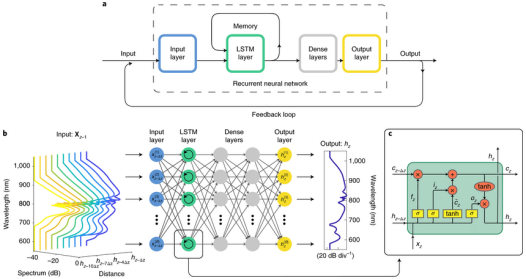


Types of Neural Networks

- **Perceptron:** The basic building block.
- **Multilayer Perceptron (MLP):** Multiple layers, capable of learning non-linear functions.
- **Convolutional Neural Networks (CNNs):** Specialized for image and video processing.
- **Recurrent Neural Networks (RNNs):** Designed for sequential data like text and time series.
- **Transformers:** Attention-based models, excelling in NLP tasks.



CNN



RNN

Applications and Benefits

Applications:

- Image Recognition: Object detection, image classification.
- Natural Language Processing: Machine translation, sentiment analysis.
- Healthcare: Disease diagnosis, drug discovery.
- Finance: Fraud detection, stock market prediction.

Benefits:

- High accuracy.
- Ability to learn complex patterns.
- Adaptability to different data types.

Neural Networks vs. Traditional ML

Feature	Neural Networks	Traditional ML
Data	Large datasets; performance scales with data.	Smaller datasets; less data-dependent.
Complexity	High; computationally intensive; GPUs often needed.	Lower; faster training; less resource-intensive.
Feature Eng.	Automatic feature learning; less manual effort.	Requires manual feature engineering.
Interpretability	"Black box"; difficult to interpret.	More interpretable (e.g., decision trees).
Generalization	Good with diverse data; prone to overfitting.	Can generalize; sensitive to feature/model choice.
Use Cases	Complex tasks (image/text/time series).	Broader range; good for limited data/interpretability.

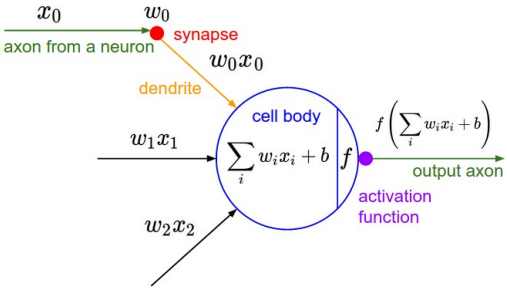
Processing Principles: Layers and Calculations

- **Input Layer:** Receives the input data.
- **Hidden Layers:** Perform computations.
- **Output Layer:** Produces the final result.
- **Activation Function:** Introduces non-linearity (e.g., sigmoid, ReLU).

$$\text{Output} = f\left(\sum_{i=1}^{n} w_i x_i + b\right)$$

where:

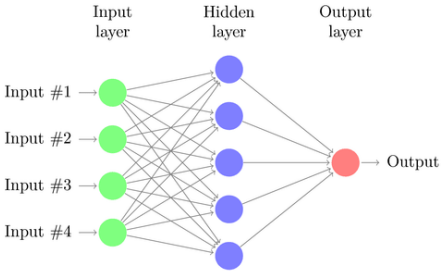
- x_i: Input values
- w_i: Weights
- b: Bias
- f: Activation function



A single neuron calculation

Simple Neural Network Architecture

- Neural networks are composed of layers of interconnected neurons.
- The basic structure includes:
  - **Input Layer:** Receives the initial data.
  - **Hidden Layer(s):** Perform computations on the input.
  - **Output Layer:** Produces the final result.



A simple neural network with layers

The Input Layer

- The first layer of the network.
- Receives raw input data as a vector.
- Each neuron in the input layer corresponds to a feature of the input data.
- No computation occurs in the input layer; it simply passes the data to the next layer.

Example: If the input is an image of size 28x28 pixels, the input layer would have 784 neurons (28 * 28).

Hidden Layers

- Layers between the input and output layers.
- Perform the core computations of the network.
- Each neuron in a hidden layer receives input from all neurons in the previous layer.
- Computations involve:
  - Weighted sum of inputs.
  - Application of an activation function.

$$\text{Output}_j = f\left(\sum_{i=1}^{n} w_{ij} x_i + b_j\right)$$

where:

- x_i: Input from the i-th neuron in the previous layer.
- w_{ij}: Weight connecting the i-th neuron in the previous layer to the j-th neuron in the current layer.
- b_j: Bias for the j-th neuron.
- f: Activation function (e.g., ReLU, sigmoid).

Activation Functions

- Introduce non-linearity into the network.
- Without activation functions, the network would only be able to learn linear relationships.
- Common activation functions:

  ReLU (Rectified Linear Unit):

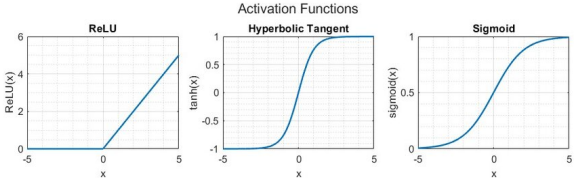
$$f(x) = \max(0, x)$$

  Sigmoid:

$$f(x) = \frac{1}{1+e^{-x}}$$

  Tanh:

$$f(x) = \frac{e^{x}-e^{-x}}{e^{x}+e^{-x}}$$



The Output Layer

- The final layer of the network.
- Produces the output of the network.
- The number of neurons in the output layer depends on the task:
  - **Classification:** One neuron per class (or a single neuron with sigmoid for binary classification).
  - **Regression:** One neuron with a linear activation function.

Output Layer: Classification

- Goal: Assign an input to one of several predefined categories or classes.
- Number of neurons: Typically one neuron per class.
- Activation function:
  - **Softmax:** Used for multi-class classification (more than two classes). It converts the output of each neuron into a probability distribution over the classes.

$$\text{Softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

    where z is the vector of outputs from the previous layer and K is the number of classes.
  - **Sigmoid:** Used for binary classification (two classes). The output is interpreted as the probability of belonging to the positive class.

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

- Output: A vector of probabilities, where each element represents the probability of the input belonging to a specific class. The class with the highest probability is the predicted class.

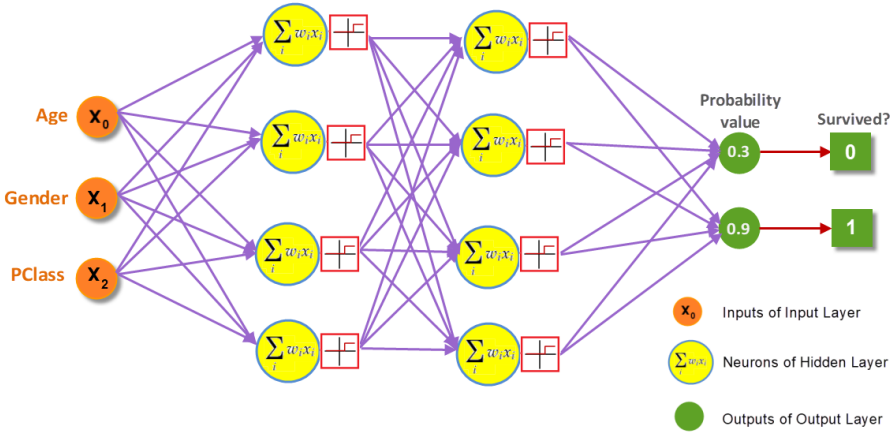
Output Layer: Regression

- Goal: Predict a continuous numerical value.
- Number of neurons: Typically one neuron.
- Activation function:
  - **Linear (Identity):** $f(x) = x$. Often used when the output can take any real value.
  - Sometimes no activation function is used at all.
- Output: A single numerical value representing the prediction.

Example: Predicting house prices, stock prices, or temperature.

- Suppose the output of the last hidden layer is 2.5.
- With a linear activation function, the output of the output neuron is also 2.5.
- This 2.5 represents the predicted value (e.g., $250,000 if predicting house prices in thousands of dollars).

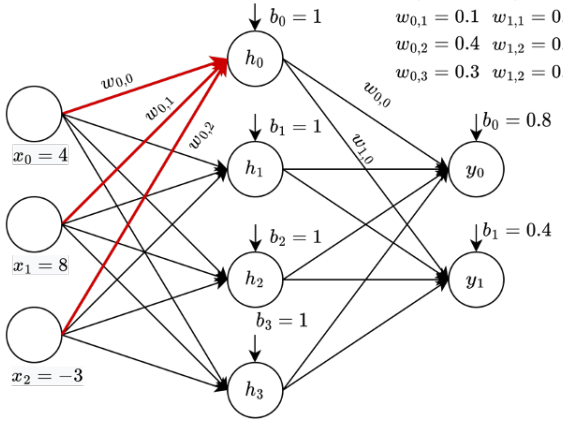
Summary


Age
Gender
PClass
X_0
X_1
X_2
Probability value
Survived?
0.3
0.9
0
1
Inputs of Input Layer
Neurons of Hidden Layer
Outputs of Output Layer

Exercises

$w_{0,0} = 0.3 \quad w_{2,0} = 0.5 \quad w_{1,0} = 0.1 \quad w_{3,0} = 0.4$
$w_{0,1} = 0.2 \quad w_{2,1} = 0.2 \quad w_{1,1} = 0.2 \quad w_{3,1} = 0.6$
$w_{0,2} = 0.4 \quad w_{2,2} = 0 \quad w_{1,2} = 0.8 \quad w_{3,2} = 0.7$

$w_{0,0} = 0.2 \quad w_{1,0} = 0.5$
$w_{0,1} = 0.1 \quad w_{1,1} = 0.9$
$w_{0,2} = 0.4 \quad w_{1,2} = 0.8$
$w_{0,3} = 0.3 \quad w_{1,2} = 0.6$



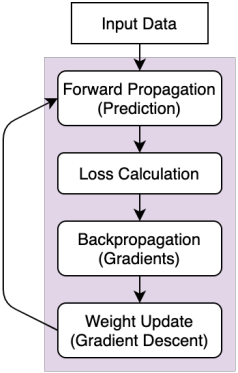
activation function: sigmoid

$$f(x) = \frac{1}{1+e^{-x}}$$

Sum calculation

$$y = \sum_{i=1}^{K} x_i w_i + b_i$$

The Training Process: Overview



Training Loop

- Training a neural network involves adjusting its weights and biases to minimize the difference between its predictions and the actual target values.
- This process is typically iterative and involves the following steps:
  1. **Forward Propagation:** Input data is passed through the network to generate predictions.
  2. **Loss Calculation:** A loss function measures the error between predictions and true values.
  3. **Backpropagation:** The error is propagated back through the network to calculate gradients (how much each weight contributes to the error).
  4. **Weight Update:** Weights and biases are adjusted based on the gradients to reduce the loss.

Forward Propagation

- Input data is fed into the input layer.
- Data is passed through each layer, with each neuron performing a weighted sum of its inputs and applying an activation function.
- This process continues until the output layer produces a prediction.

$$\text{Output}_j = f\left(\sum_{i=1}^{n} w_{ij} x_i + b_j\right)$$

Loss Functions: Measuring Error

- A loss function quantifies the difference between the network's predictions and the actual target values.
- Common loss functions:
  - **Mean Squared Error (MSE):** Used for regression.

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

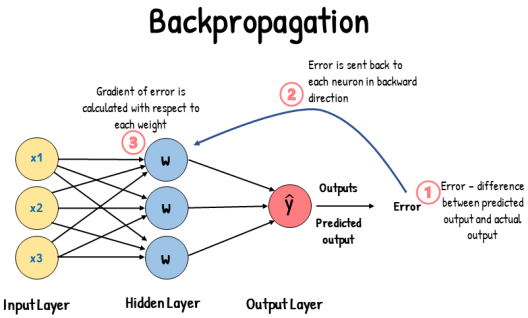
    where y_i are the true values and $\hat{y}_i$ are the predictions.
  - **Cross-Entropy Loss:** Used for classification.

$$\text{Cross-Entropy} = -\frac{1}{n}\sum_{i=1}^{n}\sum_{c=1}^{C} y_{ic}\log(\hat{y}_{ic})$$

    where y_{ic} is 1 if the i-th sample belongs to class c and 0 otherwise, and $\hat{y}_{ic}$ is the predicted probability of the i-th sample belonging to class c.

Backpropagation: Calculating Gradients

- Backpropagation calculates the gradients of the loss function with respect to each weight and bias in the network.
- These gradients indicate how much each weight contributes to the error.
- The chain rule of calculus is used to efficiently calculate these gradients through the network's layers.



Backpropagation

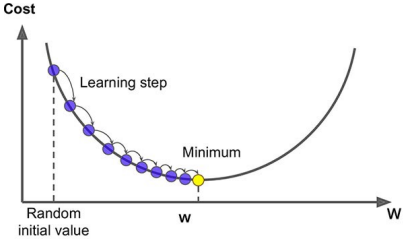
Weight Update: Gradient Descent

- Gradient descent is used to update the weights and biases based on the calculated gradients.
- The weights are adjusted in the direction that reduces the loss.
- The update rule is:

$$w_{ij} = w_{ij} - \eta \frac{\partial \text{Loss}}{\partial w_{ij}}$$

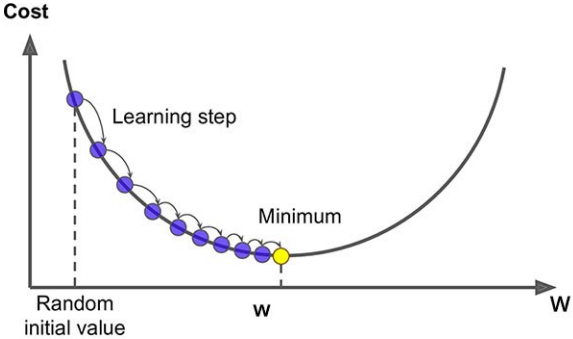
where η is the learning rate, which controls the step size of the update.

The goal is to find the **argmin** of the loss function, i.e., the weights that minimize the loss.



The Idea Behind Gradient Descent

- Goal: Find the minimum of a function (the loss function in our case).
- Analogy: Imagine you are on top of a mountain in dense fog and want to reach the valley.
- You can only see the ground immediately around you.
- Gradient descent: You take small steps in the direction of the steepest descent (the direction where the ground slopes downwards most steeply).



Mathematical Formulation

- The gradient of a function points in the direction of the steepest ascent.
- To find the minimum, we move in the **opposite** direction of the gradient.
- The update rule for a weight w is:

$$w_{new} = w_{old} - \eta \frac{\partial Loss}{\partial w}$$

where:

- η (eta) is the **learning rate**, a small positive value that controls the step size.
- $\frac{\partial Loss}{\partial w}$ is the partial derivative of the loss function with respect to w, which represents the gradient.

Example Calculation

Let's consider a simple loss function: $Loss(w) = w^2 - 4w + 5$

- We want to find the minimum value of w using gradient descent.
- The derivative (gradient) of the loss function is: $\frac{\partial Loss}{\partial w} = 2w - 4$
- Let's start with an initial guess $w = 0$ and a learning rate $\eta = 0.1$.
- **Iteration 1:**
  - Gradient: $2(0) - 4 = -4$
  - Update: $w_{new} = 0 - 0.1(-4) = 0.4$
- **Iteration 2:**
  - Gradient: $2(0.4) - 4 = -3.2$
  - Update: $w_{new} = 0.4 - 0.1(-3.2) = 0.72$
- We continue this process until the change in w becomes very small or a maximum number of iterations is reached.

Example Calculation (Cont)

- Let's continue the example with $Loss(w) = w^2 - 4w + 5$, $\frac{\partial Loss}{\partial w} = 2w - 4$, $w_{initial} = 0$, and $\eta = 0.1$.
- Here's a table showing several iterations:

Iteration	w_{old}	Gradient	w_{new}
1	0	-4	0.4
2	0.4	-3.2	0.72
3	0.72	-2.56	0.976
4	0.976	-2.048	1.1808
5	1.1808	-1.6384	1.34464
...	...	...	...
10	1.758	-0.484	1.8064
20	1.990	-0.020	1.992
50	1.99999	-0.00002	2.000

- As we can see, w converges towards 2, which is the actual minimum of the function (found by setting the derivative to 0: $2w - 4 = 0 \Rightarrow w = 2$).

Choosing the Learning Rate

- The learning rate η is a crucial hyperparameter. It controls the step size in gradient descent.
- **Too small:** Convergence is very slow, requiring many iterations to reach the minimum.
- **Too large:** The algorithm might overshoot the minimum and oscillate or even diverge, never finding a stable solution.
- **Optimal learning rate:** Finding the optimal learning rate often requires experimentation. Techniques like learning rate scheduling (adjusting the learning rate during training) can be helpful.
- **Learning Rate Scheduling:** Adjusting the learning rate during training. Common approaches:
  - **Step Decay:** Reduce the learning rate by a fixed factor after a certain number of epochs.
  - **Exponential Decay:** Reduce the learning rate exponentially over time.

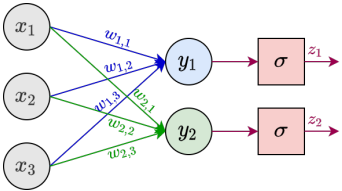
Simple Neural Network Setup

- Consider a simple neural network with 3 inputs (x_1, x_2, x_3), one hidden layer with 2 neurons, and one output neuron.
- We have a dataset of 10 data points $\{x_1, x_2, x_3\}$.
- The output of the hidden neurons is calculated as:

$$y_1 = x_1 w_{1,1} + x_2 w_{1,2} + x_3 w_{1,3} + b_1$$
$$y_2 = x_1 w_{2,1} + x_2 w_{2,2} + x_3 w_{2,3} + b_2$$

  Where $w_{i,j}$ are the weights connecting input j to hidden neuron i, and b_i are the biases.
- The output of the network z_1 is calculated using a sigmoid activation function:

$$z_1 = \sigma(y_1) = \frac{1}{1 + e^{-y_1}}$$



Loss Function

- We use the Mean Squared Error (MSE) as our loss function:

$$\mathcal{L} = \frac{1}{10}\sum_{k=1}^{10}(o_1^k - z_1^k)^2$$

where o_1^k is the desired output (ground truth) for the k-th data point, and z_1^k is the network's output for the k-th data point.

Backpropagation: Calculating the Gradient

- We want to update the weights using gradient descent:

$$w_{j,i} \leftarrow w_{j,i} - \eta \frac{\partial \mathcal{L}}{\partial w_{j,i}}$$

  where η is the learning rate.

- Using the chain rule, we can calculate the partial derivative of the loss with respect to a weight $w_{j,i}$:

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial w_{j,i}} &= \frac{\partial \mathcal{L}}{\partial z_1} \frac{\partial z_1}{\partial y_1} \frac{\partial y_1}{\partial w_{j,i}} \\ &= \frac{2}{10} \sum_{k=1}^{10} (z_1^k - o_1^k) \cdot z_1^k (1 - z_1^k) \cdot x_i^k\end{aligned}$$

Weight Update

- Finally, we update the weights:

$$w_{j,i} \leftarrow w_{j,i} - \eta \left(\frac{2}{10} \sum_{k=1}^{10} (z_1^k - o_1^k) \cdot z_1^k (1 - z_1^k) \cdot x_i^k \right)$$

- This process is repeated for all weights in the network for each training example (or batch of examples) until the loss converges to a minimum.

Python Code - Hand-writing Digit Classification

```
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt

print('load data from MNIST')
mnist = tf.keras.datasets.mnist
(x_train, y_train), (x_test, y_test) = mnist.load_data()

# Use all digits 0-9
x_train = x_train.astype(np.float32)
x_test = x_test.astype(np.float32)

y_train = tf.keras.utils.to_categorical(y_train, num_classes=10)
y_test = tf.keras.utils.to_categorical(y_test, num_classes=10)

# Reshape and normalize
x_train = np.reshape(x_train, (-1, 784)) / 255.0
x_test = np.reshape(x_test, (-1, 784)) / 255.0
```

Python Code - Hand-writing Digit Classification

```
def sigmoid(x):
    return 1.0 / (1.0 + np.exp(-x))

# Initialize weights
W = np.random.uniform(-0.1, 0.1, (784, 10))

# Training process
n = 0.05  # Learning rate
num_epoch = 10  # Number of epochs
batch_size = 32
losses = []
train_accuracies = []
test_accuracies = []
```

Python Code - Hand-writing Digit Classification

```
for epoch in range(num_epoch):
    for batch_start in range(0, x_train.shape[0], batch_size):
        batch_end = min(batch_start + batch_size, x_train.shape[0])
        x_batch = x_train[batch_start:batch_end]
        y_batch = y_train[batch_start:batch_end]

        o = sigmoid(np.matmul(x_batch, W))
        dW = (x_batch.T @ ((y_batch - o) * o * (1 - o))) / batch_size
        W = W + n * dW

    # Calculate and store loss and accuracy for the full training set
    train_o = sigmoid(np.matmul(x_train, W))
    loss = np.mean(np.power(train_o - y_train, 2))
    losses.append(loss)

    train_predictions = np.argmax(train_o, axis=1)
    train_true_labels = np.argmax(y_train, axis=1)
    train_accuracy = np.mean(train_predictions == train_true_labels)
    train_accuracies.append(train_accuracy)

    # Calculate test accuracy
    test_o = sigmoid(np.matmul(x_test, W))
    test_predictions = np.argmax(test_o, axis=1)
    test_true_labels = np.argmax(y_test, axis=1)
    test_accuracy = np.mean(test_predictions == np.argmax(y_test, axis=1))
    test_accuracies.append(test_accuracy)

    print(f"Epoch {epoch+1}/{num_epoch}, Loss: {loss}, Train Accuracy: {train_accuracy}, Test Accuracy: {test_accuracy}")
```

Python Code - Hand-writing Digit Classification

```
# Plotting the results
epochs = range(1, num_epoch + 1)
plt.figure(figsize=(12, 4))

plt.subplot(1, 3, 1)
plt.plot(epochs, losses)
plt.title("Loss vs. Epoch")
plt.xlabel("Epoch")
plt.ylabel("Loss")

plt.subplot(1, 3, 2)
plt.plot(epochs, train_accuracies, label='Training Accuracy')
plt.title("Training Accuracy vs. Epoch")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")

plt.subplot(1, 3, 3)
plt.plot(epochs, test_accuracies, label='Test Accuracy', color='orange')
plt.title("Test Accuracy vs. Epoch")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.tight_layout()
plt.show()

# Example of getting predictions (on test data)
test_o = sigmoid(np.matmul(x_test, W))
predictions = np.argmax(test_o, axis=1)
print("Example Predictions (first 10 test samples):", predictions[:10])
```

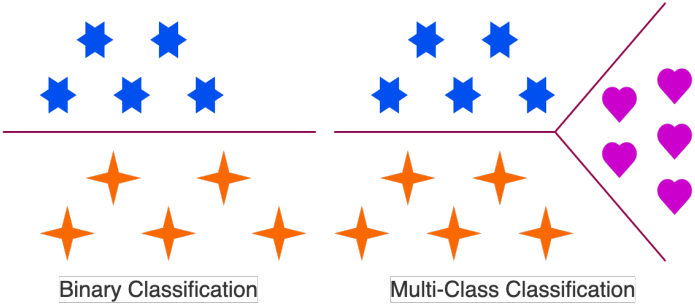
Student Assignment: Code Understanding & Analysis

Objective: To understand the practical implementation of a simple neural network.

- **Code Dissection:** Students will meticulously analyze the provided Python code, explaining the purpose of each line and its role within the overall machine learning framework. This includes:
- **Architectural Exploration**: Students are challenged to explore architectural variations of the simple neural network by experimenting with additional hidden layers and varying neuron counts.

Introduction to Classification

- **Classification:** A supervised learning task where the goal is to assign input data to predefined categories or classes.
- Two main types:
  - **Binary Classification:** Two classes (e.g., spam/not spam, cat/dog).
  - **Multi-class Classification:** More than two classes (e.g., handwritten digits 0-9, types of flowers).



Binary Classification

- **Definition:** A classification task with two possible outcomes or classes.
- **Representation:**
  - Often encoded as 0 and 1 (or True/False).
  - Sometimes represented as -1 and 1, especially in certain algorithms like Support Vector Machines (SVMs).
- **Decision Boundary:** The goal is to find a decision boundary that separates the two classes.
- **Examples:**
  - Medical diagnosis (disease present/absent).
  - Fraud detection (fraudulent/not fraudulent).
  - Sentiment analysis (positive/negative).

Binary Classification

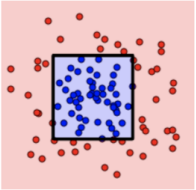
- **Probabilistic Interpretation:** Many binary classifiers output a probability score between 0 and 1, representing the likelihood of belonging to one of the classes.
- **Thresholding:** A threshold (typically 0.5) is used to make a final classification decision:
  - Probability $\geq$ threshold: Class 1
  - Probability ¡ threshold: Class 0
- **Examples:**
  - Medical diagnosis (probability of disease ¿ 0.7: disease present).
  - Fraud detection (probability of fraud ¿ 0.9: fraudulent).
  - Sentiment analysis (probability of positive sentiment ¿ 0.6: positive).

Binary Classification

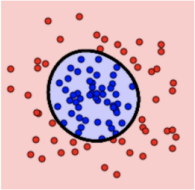
- **Loss Functions:** Used to train binary classifiers by measuring the difference between predicted and actual labels.
- **Common Loss Functions:**
  - Binary Cross-Entropy (Log Loss)
  - Hinge Loss (used in SVMs)
- **Examples:**
  - Medical diagnosis (minimizing false negatives).
  - Fraud detection (minimizing false positives).
  - Sentiment analysis (maximizing accuracy).

Binary Classification

- **Two Outcomes:** The goal is to classify data into one of two distinct categories.
- **Probabilities and Thresholds:** Classifiers often output a probability $P(y = 1|x)$, representing the likelihood of belonging to class 1. A threshold (e.g., 0.5) determines the final classification.
- **Decision Boundary:** A line or surface that separates the two classes in the feature space.



Decision Tree



Support Vector Machine

Example of a Decision Boundary

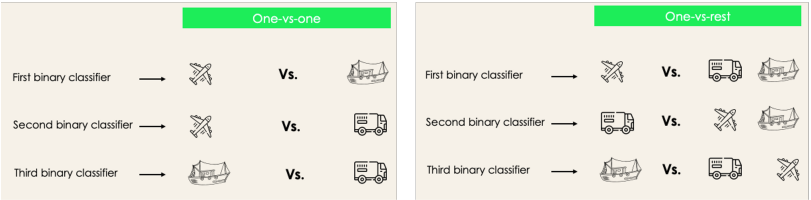
- **Examples:**
  - Medical diagnosis (disease present/absent).
  - Fraud detection (fraudulent/not fraudulent).
  - Sentiment analysis (positive/negative).

Multi-class Classification

- **Definition:** A classification task where the goal is to assign input data to one of *more than two* predefined categories or classes.
- **Output Representation:**
  - Often represented using one-hot encoding: a vector where each element corresponds to a class, and only one element is 1 (representing the predicted class), while the others are 0.
  - Example: For 3 classes (cat, dog, bird), a prediction of "dog" could be represented as [0, 1, 0].
- **Challenges:**
  - Increased complexity compared to binary classification.
  - Need for specialized algorithms and techniques (e.g., softmax regression, one-vs-rest, one-vs-one).
- **Examples:**
  - Image classification (cat, dog, bird, fish, etc.).
  - Handwritten digit recognition (0-9).
  - News categorization (sports, politics, technology, business, etc.).
  - Object recognition in images (car, pedestrian, traffic light, building, etc.).

Strategies for Multi-class Classification

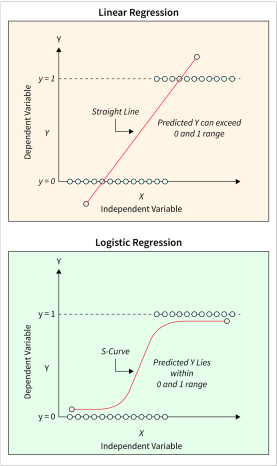
- **One-vs-Rest (OvR) / One-vs-All (OvA):** Train one binary classifier for each class, treating that class as "positive" and all other classes as "negative." For n classes, you need n classifiers.
- **One-vs-One (OvO):** Train a binary classifier for each pair of classes. For n classes, you train $\frac{n(n-1)}{2}$ classifiers.



One-vs-one and one-vs-rest

Logistic Regression

- Used for binary classification.
- Combines linear regression with a sigmoid activation function.
- Linear Regression: $y = w^T x + b$
- Sigmoid Function: $\sigma(y) = \frac{1}{1+e^{-y}}$
- Logistic Regression: $P(y = 1|x) = \sigma(w^T x + b)$ (Probability of belonging to class 1)



Logistic regression

Softmax Regression

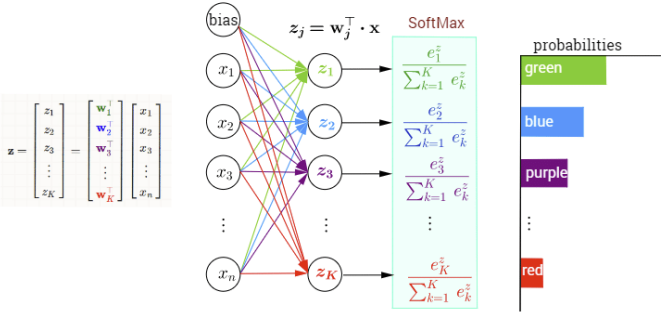
- Generalization of logistic regression to multi-class classification.
- Calculates the probability of each class.
- Softmax function:

$$P(y = i|x) = \frac{e^{w_i^T x + b_i}}{\sum_{j=1}^{K} e^{w_j^T x + b_j}}$$

where K is the number of classes.

Softmax in Neural Networks

- In neural networks, the softmax function is often used in the output layer for multi-class classification.
- The output of the softmax layer is a probability distribution over the classes.
- The class with the highest probability is the predicted class.

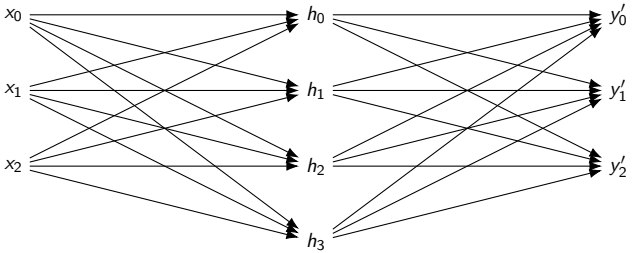


Softmax in neural networks for multi-class classification

Network Architecture

Network configuration:

- 3 input nodes (x_0, x_1, x_2).
- 4 hidden neurons (h_0, h_1, h_2, h_3) with sigmoid activation.
- 3 output neurons (y'_0, y'_1, y'_2) with softmax activation.
- Cross-entropy loss function.



Architecture of the three-layer neural network.

Forward Propagation and Loss

- Hidden layer activations:

$$h_j = \sigma\left(\sum_{i=0}^{2} w_{ji}^{(1)} x_i + b_j^{(1)}\right), \quad j = 0, 1, 2, 3$$

  where $\sigma(z) = \frac{1}{1+e^{-z}}$ (sigmoid).

- Output layer activations (softmax):

$$y'_k = \frac{e^{\sum_{j=0}^{3} w_{kj}^{(2)} h_j + b_k^{(2)}}}{\sum_{l=0}^{2} e^{\sum_{j=0}^{3} w_{lj}^{(2)} h_j + b_l^{(2)}}}, \quad k = 0, 1, 2$$

Setup and Definitions

- We consider a neural network with a softmax output layer and cross-entropy loss.
- Cross-Entropy Loss (single example):

$$L = -\sum_{k=0}^{2} y_k \log(y'_k)$$

where y_k is true label (one-hot encoded) and y'_k denotes softmax output (predicted probability for class k).

- Softmax Function:

$$y'_k = \frac{e^{z_k}}{\sum_{l=0}^{2} e^{z_l}}$$

where $z_k = \sum_{j=0}^{3} w_{kj}^{(2)} h_j + b_k^{(2)}$ (weighted sum of inputs to output neuron k).

Applying the Chain Rule

- We want to find the derivative of the loss with respect to the output layer weights, $\frac{\partial L}{\partial w_{kj}^{(2)}}$.
- Using the chain rule:

$$\frac{\partial L}{\partial w_{kj}^{(2)}} = \frac{\partial L}{\partial y'_k} \cdot \frac{\partial y'_k}{\partial z_k} \cdot \frac{\partial z_k}{\partial w_{kj}^{(2)}}$$

- Derivative of Loss with respect to Softmax Output:

$$\frac{\partial L}{\partial y'_k} = -\frac{y_k}{y'_k}$$

- Derivative of Softmax Output with respect to z_k:
  - If $k = l$: $\frac{\partial y'_k}{\partial z_k} = y'_k(1 - y'_k)$
  - If $k \neq l$: $\frac{\partial y'_k}{\partial z_l} = -y'_k y'_l$

Calculating the Partial Derivatives

- The derivative of the Loss with respect to z_k is:

$$
\begin{aligned}
\frac{\partial L}{\partial z_k} &= \sum_{l=0}^{2} \frac{\partial L}{\partial y'_l} \frac{\partial y'_l}{\partial z_k} \\
&= \frac{\partial L}{\partial y'_k} \frac{\partial y'_k}{\partial z_k} + \sum_{l \neq k} \frac{\partial L}{\partial y'_l} \frac{\partial y'_l}{\partial z_k} \\
&= \left(-\frac{y_k}{y'_k}\right) y'_k (1 - y'_k) + \sum_{l \neq k} \left(-\frac{y_l}{y'_l}\right) (-y'_k y'_l) \\
&= -y_k (1 - y'_k) + y'_k \sum_{l \neq k} y_l \\
&= -y_k + y_k y'_k + y'_k (1 - y_k) = y'_k - y_k
\end{aligned}
$$

Final Result

- Derivative of z_k with respect to $w_{kj}^{(2)}$:

$$\frac{\partial z_k}{\partial w_{kj}^{(2)}} = h_j$$

- Combining all parts of the chain rule:

$$\frac{\partial L}{\partial w_{kj}^{(2)}} = \frac{\partial L}{\partial y_k'} \cdot \frac{\partial y_k'}{\partial z_k} \cdot \frac{\partial z_k}{\partial w_{kj}^{(2)}} = (y_k' - y_k)h_j$$

- This is the gradient we use to update the weights in the output layer during backpropagation.

$$w_{kj}^{(2)} \leftarrow w_{kj}^{(2)} - \eta(y_k' - y_k)h_j$$

where η is the learning rate.

Backpropagation: Hidden Layer Weights

- Derivative of loss with respect to hidden layer weights $w_{ji}^{(1)}$:

$$\frac{\partial L}{\partial w_{ji}^{(1)}} = \left(\sum_{k=0}^{2} (y'_k - y_k) w_{kj}^{(2)} \right) h_j (1 - h_j) x_i$$

- Weight update rule:

$$w_{ji}^{(1)} \leftarrow w_{ji}^{(1)} - \eta \left(\sum_{k=0}^{2} (y'_k - y_k) w_{kj}^{(2)} \right) h_j (1 - h_j) x_i$$

Chain Rule for Hidden Layer Weights

- We use the chain rule again, but this time it's more complex because the hidden neurons affect *all* output neurons:

$$\frac{\partial L}{\partial w_{ji}^{(1)}} = \frac{\partial L}{\partial h_j} \cdot \frac{\partial h_j}{\partial w_{ji}^{(1)}}$$

Calculating the Partial Derivatives

- The activation of the hidden neuron h_j is given by:

$$h_j = \sigma(z_j) = \sigma\left(\sum_{i=0}^{2} w_{ji}^{(1)} x_i + b_j^{(1)}\right)$$

- Therefore:

$$\frac{\partial h_j}{\partial w_{ji}^{(1)}} = \frac{\partial \sigma(z_j)}{\partial z_j} \cdot \frac{\partial z_j}{\partial w_{ji}^{(1)}} = h_j(1 - h_j)x_i$$

Derivative of the Sigmoid Function

- Let's first find the derivative of the sigmoid function $\sigma(z)$ with respect to z:

$$\begin{aligned}
\frac{\partial \sigma(z)}{\partial z} &= \frac{\partial}{\partial z}\left(\frac{1}{1+e^{-z}}\right) \\
&= \frac{0-(-1)e^{-z}}{(1+e^{-z})^2} = \frac{e^{-z}}{(1+e^{-z})^2} \\
&= \frac{1}{1+e^{-z}} \cdot \frac{e^{-z}}{1+e^{-z}} \\
&= \frac{1}{1+e^{-z}} \cdot \left(1 - \frac{1}{1+e^{-z}}\right) \\
&= \sigma(z)(1-\sigma(z))
\end{aligned}$$

- Therefore:

$$\frac{\partial \sigma(z_j)}{\partial z_j} = h_j(1-h_j)$$

Calculating the Partial Derivatives

- This is the crucial part. The hidden neuron h_j influences *all* output neurons. Therefore, we need to sum over the contributions from each output neuron:

$$
\begin{aligned}
\frac{\partial L}{\partial h_j} &= \sum_{k=0}^{2} \frac{\partial L}{\partial y'_k} \cdot \frac{\partial y'_k}{\partial h_j} \\
&= \sum_{k=0}^{2} \frac{\partial L}{\partial y'_k} \cdot \frac{\partial y'_k}{\partial z_k} \cdot \frac{\partial z_k}{\partial h_j} \\
&= \sum_{k=0}^{2} (y'_k - y_k) w_{kj}^{(2)}
\end{aligned}
$$

- Here, $\frac{\partial z_k}{\partial h_j} = w_{kj}^{(2)}$

Putting It All Together

- Combining the results:

$$\begin{aligned}\frac{\partial L}{\partial w_{ji}^{(1)}} &= \frac{\partial L}{\partial h_j} \cdot \frac{\partial h_j}{\partial w_{ji}^{(1)}} \\ &= \left(\sum_{k=0}^{2} (y'_k - y_k) w_{kj}^{(2)} \right) h_j (1 - h_j) x_i\end{aligned}$$

- This is the gradient used to update the weights between the input and hidden layers.

Batch vs. Mini-Batch Gradient Descent

Two main variants differing in how much training data is used to compute the gradient:

- **Batch Gradient Descent:**
  - Uses the *entire* training dataset to compute the gradient of the loss function in each iteration (epoch).
  - Computationally expensive for large datasets.
  - Provides a more stable but potentially slower convergence.
  - Update rule (for a weight w): $w_{new} = w_{old} - \eta \nabla L(w; \text{all data})$
- **Mini-Batch Gradient Descent:**
  - Uses a small random subset (a "mini-batch") of the training data to compute the gradient in each iteration.
  - More computationally efficient, especially for large datasets.
  - Introduces some noise in the gradient estimation, which can help escape local minima and potentially lead to faster convergence.
  - Update rule (for a weight w): $w_{new} = w_{old} - \eta \nabla L(w; \text{mini-batch})$

Stochastic Gradient Descent (SGD): A special case of mini-batch gradient descent where the mini-batch size is 1 (i.e., each iteration uses only one training example).

Python Code

Students will work with a simple neural network implemented from scratch (without using high-level APIs like TensorFlow or PyTorch for training). The code provided demonstrates key steps of training a neural network, including forward propagation, loss computation, backpropagation, and weight updates. The task involves analyzing, modifying, and extending the code to deepen understanding of neural network mechanics.

Learning Objectives

- Understand the role of forward and backward propagation in neural networks.
- Learn how to compute gradients and update weights manually.
- Explore the effect of hyperparameters such as learning rate and number of hidden units.
- Analyze the training process by monitoring loss and accuracy.

Python Code - Batch GD

```
import numpy as np
import tensorflow as tf
#load datashet
print("Load MNIST Database")
mnist = tf.keras.datasets.mnist
(x_train,y_train),(x_test,y_test)= mnist.load_data()
x_train=np.reshape(x_train,(60000,784))/255.0
x_test= np.reshape(x_test,(10000,784))/255.0
y_train = np.matrix(np.eye(10)[y_train])
y_test = np.matrix(np.eye(10)[y_test])
print("----------------------------------")
print(x_train.shape)
print(y_train.shape)

def sigmoid(x):
    return 1./(1.+np.exp(-x))
```

Python Code - Batch GD

```
def softmax(x):
    return np.divide(np.matrix(np.exp(x)),np.mat(np.sum(np.exp(x),axis=1)))
def Forwardpass(X,Wh,bh,Wo,bo):
    zh = X@Wh.T + bh
    a = sigmoid(zh)
    z=a@Wo.T + bo
    o = softmax(z)
    return o
def AccTest(label,prediction):    # calculate the matching score
    OutMaxArg=np.argmax(prediction,axis=1)
    LabelMaxArg=np.argmax(label,axis=1)
    Accuracy=np.mean(OutMaxArg==LabelMaxArg)
    return Accuracy
learningRate = 0.5
Epoch=50
NumTrainSamples=60000
NumTestSamples=10000
```

Python Code - Batch GD

```
NumInputs=784
NumHiddenUnits=512
NumClasses=10
#inital weights
#hidden layer
Wh=np.matrix(np.random.uniform(-0.5,0.5,(NumHiddenUnits,NumInputs)))
bh= np.random.uniform(0,0.5,(1,NumHiddenUnits))
dWh= np.zeros((NumHiddenUnits,NumInputs))
dbh= np.zeros((1,NumHiddenUnits))
#Output layer
Wo=np.random.uniform(-0.5,0.5,(NumClasses,NumHiddenUnits))
bo= np.random.uniform(0,0.5,(1,NumClasses))
dWo= np.zeros((NumClasses,NumHiddenUnits))
dbo= np.zeros((1,NumClasses))
```

Python Code - Batch GD

```
from IPython.display import clear_output
loss = []
Acc = []
for ep in range (Epoch):
    #feed fordware propagation
    x = x_train
    y=y_train
    zh = x@Wh.T + bh
    a = sigmoid(zh)
    z=a@Wo.T + bo
    o = softmax(z)
    loss.append(-np.sum(np.multiply(y,np.log10(o))))
    d = o-y
    dh = d@Wo
    dhs = np.multiply(np.multiply(dh,a),(1-a))
```

Python Code - Batch GD

```
for ep in range (Epoch):
    ...

    #update weight
    dWo = np.matmul(np.transpose(d),a)
    dbo = np.mean(d)   # consider a is 1 for bias
    dWh = np.matmul(np.transpose(dhs),x)
    dbh = np.mean(dhs) # consider a is 1 for bias
    Wo =Wo - learningRate*dWo/NumTrainSamples
    bo =bo - learningRate*dbo
    Wh =Wh-learningRate*dWh/NumTrainSamples
    bh =bh-learningRate*dbh
    #Test accuracy with random innitial weights
    prediction = Forwardpass(x_test,Wh,bh,Wo,bo)
    Acc.append(AccTest(y_test,prediction))
    clear_output(wait=True)
    plt.plot([i for i, _ in enumerate(Acc)],Acc,'o')
    plt.show()
```

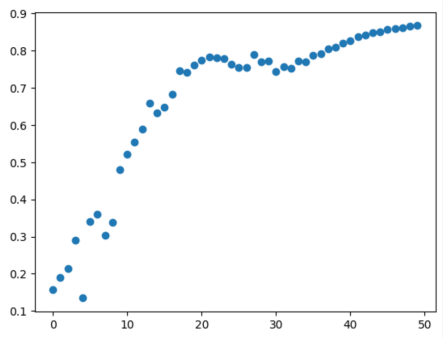
Python Code - Mini-Batch GD

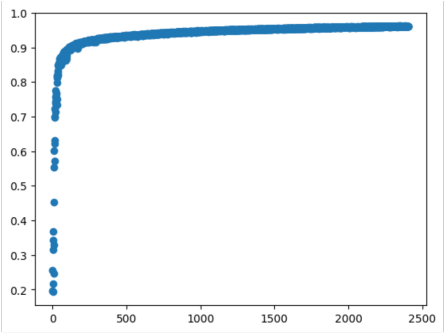
```
from IPython.display import clear_output
loss = []
Acc = []
Batch_size = 200
Stochastic_samples = np.arange(NumTrainSamples)
for ep in range (Epoch):
    np.random.shuffle(Stochastic_samples)
    for ite in range (0,NumTrainSamples,Batch_size):
        Batch_samples = Stochastic_samples[ite:ite+Batch_size]
        x = x_train[Batch_samples,:]
        y=y_train[Batch_samples,:]
        zh = x@Wh.T + bh
        a = sigmoid(zh)
        z=a@Wo.T + bo
        o = softmax(z)
        loss.append(-np.sum(np.multiply(y,np.log10(o))))
        d = o-y
        dh = d@Wo
        dhs = np.multiply(np.multiply(dh,a),(1-a))
```

Python Code - Mini-Batch GD

```
...
#update weight
    dWo = np.matmul(np.transpose(d),a)
    dbo = np.mean(d)  # consider a is 1 for bias
    dWh = np.matmul(np.transpose(dhs),x)
    dbh = np.mean(dhs)  # consider a is 1 for bias
    Wo =Wo - learningRate*dWo/Batch_size
    bo =bo - learningRate*dbo
    Wh =Wh-learningRate*dWh/Batch_size
    bh =bh-learningRate*dbh
    #Test accuracy with random innitial weights
    prediction = Forwardpass(x_test,Wh,bh,Wo,bo)
    Acc.append(AccTest(y_test,prediction))
    clear_output(wait=True)
    plt.plot([i for i, _ in enumerate(Acc)],Acc,'o')
    plt.show()
print('Epoch:', ep )
print('Accuracy:',AccTest(y_test,prediction) )
```

Batch vs Mini-Batch GD





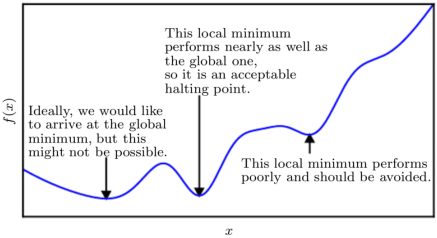
Training processing: Batch vs Mini-batch gradient descent strategies.

Assignment: Handwritten Digit Classification with Neural Networks

- **Objective:** Develop a Python program to classify handwritten digits from the MNIST dataset.
- **Model:** A four-layer neural network (Multi-Layer Perceptron):
  - Input Layer (784 neurons)
  - Hidden Layer 1 (ReLU activation)
  - Hidden Layer 2 (Sigmoid activation)
  - Output Layer (Softmax activation)
- **Training:** Implement the following:
  - Feedforward propagation
  - Backpropagation
  - Mini-batch gradient descent (batch size: 1000)
- **Submission:** Submit your Python code on Google Classroom by the deadline.

Local Minima

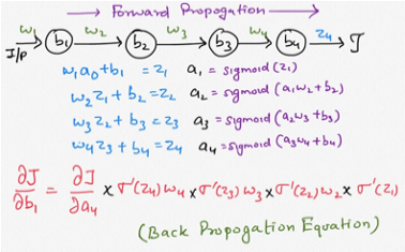
- **Challenge:** The loss function in deep learning is often non-convex, meaning it has multiple minima.
- **Local Minima:** Points where the loss is lower than in the surrounding region, but not the absolute lowest (global minimum).
- **Impact:** Gradient descent can get stuck in local minima, preventing the model from achieving optimal performance.
- **Mitigation:** Careful initialization, using larger batch sizes, and advanced optimization algorithms (like momentum) can help escape local minima.



A function with local minima

Vanishing Gradients

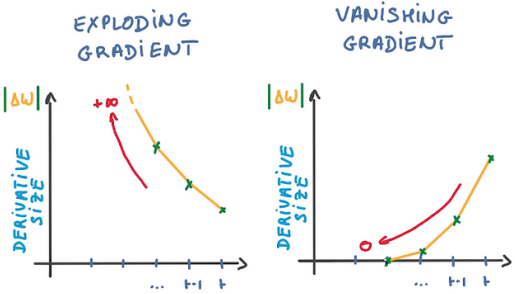
- **Challenge:** During backpropagation, gradients are multiplied as they are passed through layers.
- **Vanishing Gradients:** In deep networks, these repeated multiplications can cause gradients to become extremely small, effectively stopping the training of earlier layers.
- **Cause:** Primarily due to activation functions with derivatives close to zero in certain regions (e.g., sigmoid saturation).
- **Mitigation:** Using ReLU or its variants (Leaky ReLU, ELU), careful initialization, and batch normalization can help.



Illustrating vanishing gradients

Exploding Gradients

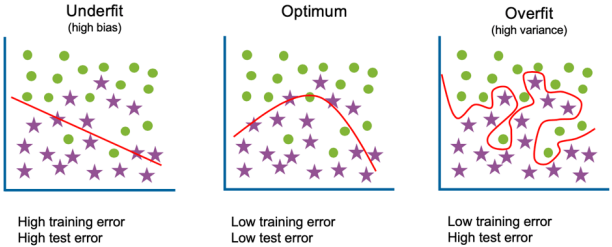
- **Challenge:** Opposite of vanishing gradients.
- **Exploding Gradients:** Gradients become extremely large during backpropagation, causing unstable training.
- **Cause:** Large weights, especially in recurrent neural networks (RNNs).
- **Mitigation:** Gradient clipping (limiting the magnitude of gradients) and weight regularization can help.



Comparing vanishing gradient and exploding gradient problems

Overfitting vs. Underfitting

- **Underfitting:** Model is too simple to capture the underlying patterns in the data. High bias.
- **Overfitting:** Model learns the training data too well, including noise, and generalizes poorly to unseen data. High variance.
- **Mitigation:**
  - **Underfitting:** Increase model complexity, train for longer.
  - **Overfitting:** Use more data, regularization (L1, L2), dropout, early stopping.



Overfitting and underfitting

Gradient Descent (Review)

- **Basic Idea:** Iteratively adjust weights to minimize the loss function.
- **Update Rule:**

$$w_{t+1} = w_t - \eta \nabla L(w_t)$$

  where:
  - w_t: Weights at time step t.
  - η: Learning rate (controls step size).
  - $\nabla L(w_t)$: Gradient of the loss function at w_t (direction of steepest ascent).
- **Limitation:** Can be slow, especially with complex loss landscapes or large datasets.

Momentum

- **Idea:** Add a "velocity" term to accelerate learning in the right direction and dampen oscillations.
- **Update Rules:**

$$v_{t+1} = \mu v_t + \eta \nabla L(w_t)$$
$$w_{t+1} = w_t - v_{t+1}$$

  where:
  - v_t: Velocity at time step t (accumulated gradient).
  - μ: Momentum term (typically 0.9). Controls the contribution of previous gradients.
- **Benefit:** Helps overcome small local minima and speeds up convergence.

Adagrad (Adaptive Gradient Algorithm)

- **Idea:** Adapt the learning rate for each parameter individually based on the historical sum of squared gradients.
- **Update Rules:**

$$G_{t+1} = G_t + (\nabla L(w_t))^2$$
$$w_{t+1} = w_t - \frac{\eta}{\sqrt{G_{t+1}} + \epsilon} \nabla L(w_t)$$

  where:
  - G_t: Sum of squared gradients up to time step t.
  - ϵ: Small constant (e.g., 10^{-8}) for numerical stability (prevents division by zero).
- **Benefit:** Well-suited for sparse data. Adapts learning rates for different features.
- **Limitation:** Accumulating gradients can cause the learning rate to become very small, stopping learning.

RMSProp (Root Mean Square Propagation)

- **Idea:** Improves Adagrad by using an exponentially decaying average of squared gradients.
- **Update Rules:**

$$S_{t+1} = \rho S_t + (1-\rho)(\nabla L(w_t))^2$$
$$w_{t+1} = w_t - \frac{\eta}{\sqrt{S_{t+1}} + \epsilon} \nabla L(w_t)$$

  where:
  - S_t: Exponential moving average of squared gradients.
  - ρ: Decay rate (typically 0.9). Controls the weighting of recent vs. past gradients.
  - ϵ: Small constant for numerical stability.
- **Benefit:** Addresses Adagrad's diminishing learning rate problem.

Adam (Adaptive Moment Estimation)

- **Idea:** Combines momentum and RMSProp with gradients' first and second moments.
- **Update Rules:**

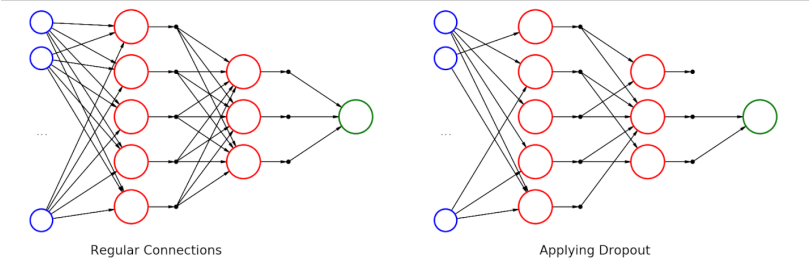
$$
\begin{aligned}
m_{t+1} &= \beta_1 m_t + (1-\beta_1)\nabla L(w_t) \\
v_{t+1} &= \beta_2 v_t + (1-\beta_2)(\nabla L(w_t))^2 \\
\hat{m}_{t+1} &= \frac{m_{t+1}}{1-\beta_1^{t+1}} \quad \text{(Bias correction)} \\
\hat{v}_{t+1} &= \frac{v_{t+1}}{1-\beta_2^{t+1}} \quad \text{(Bias correction)} \\
w_{t+1} &= w_t - \frac{\eta}{\sqrt{\hat{v}_{t+1}}+\epsilon}\hat{m}_{t+1}
\end{aligned}
$$

where:

- m_t: Exponential moving average of gradients (first moment).
- v_t: Exponential moving average of squared gradients (second moment).
- β_1, β_2: Exponential decay rates (typically 0.9 and 0.999, respectively).
- ϵ: Small constant for numerical stability.

Dropout Technique

- **Dropout:** Randomly "drops out" (sets to zero) some neurons during training.
- **Effect:** Prevents complex co-adaptations between neurons, reducing overfitting.
- **During Testing:** Neurons are not dropped out, but their activations are scaled by the dropout rate.



Illustrating dropout scheme.